

QUESTION 21

Aim

To estimate the population mean of rare-element concentration using point estimation and calculate a confidence interval.

Algorithm

1. Import NumPy and SciPy.
2. Read rare_elements.csv.
3. Input sample size, confidence level and desired precision.
4. Select a random sample.
5. Calculate the sample mean and standard deviation.
6. Calculate the confidence interval.
7. Calculate the margin of error.
8. Compare the margin of error with the desired precision.
9. Display the result.

Code

```
import numpy as np
from scipy import stats

data = np.loadtxt(
    "rare_elements.csv",
    delimiter=",",
    skiprows=1
)
n = int(input("Enter sample size: "))
confidence = float(input("Enter confidence level: "))
precision = float(input("Enter desired precision: "))

sample = np.random.choice(data, n, replace=False)
mean = np.mean(sample)
std = np.std(sample, ddof=1)
alpha = 1 - confidence / 100

t = stats.t.ppf(
    1 - alpha / 2,
    n - 1
)

margin = t * std / np.sqrt(n)

lower = mean - margin
upper = mean + margin

print("Sample Mean:", mean)
print("Confidence Interval:",
      lower, "to", upper)

print("Margin of Error:", margin)

if margin <= precision:
    print("Desired precision is achieved")
else:
    print("Desired precision is not achieved")
```

Output

```
pip3 > -
1 import numpy as np
2 from scipy import stats
3
4 data = np.loadtxt("rare_elements.csv", delimiter=",", skiprows=1)
5
6 n = int(input("Enter sample size: "))
7 confidence = float(input("Enter confidence level (e.g., 95): "))
8 precision = float(input("Enter desired precision: "))
9
10 sample = np.random.choice(data, n, replace=False)
11
12 mean = np.mean(sample)
13 std = np.std(sample, ddof=1)
14
15 alpha = 1 - confidence / 100
16 t_value = stats.t.ppf(1 - alpha / 2, n - 1)
17
18 margin = t_value * std / np.sqrt(n)
19 lower = mean - margin
20 upper = mean + margin
21
22 print("Sample Mean:", mean)
23 print("Confidence Interval:", lower, "to", upper)
24 print("Margin of Error:", margin)
25
26 if margin <= precision:
27     print("Desired precision is achieved.")
28 else:
29     print("Desired precision is not achieved.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@Unknown_e6:41:66:26:88:ae FSD LA % python3 p2.py
Rating
0 4.5
1 4.0
2 3.5
3 5.0
4 4.2
Enter confidence level (e.g., 95): 95
Average Rating: 4.313333333333333
Confidence Interval: 4.15228183868401 to 4.474384827982657
Customer satisfaction is High
```

Result

The population mean was estimated successfully using point estimation and a confidence interval.

QUESTION 22

Aim

To calculate a confidence interval for the true population mean customer rating and determine the customer satisfaction level.

Algorithm

1. Import Pandas, NumPy and SciPy.
2. Read customer_reviews.csv.
3. Calculate the mean rating.
4. Calculate standard deviation and sample size.
5. Input the confidence level.
6. Calculate the t-value and margin of error.
7. Calculate the confidence interval.
8. Determine customer satisfaction.
9. Display the results.

Code

```
import pandas as pd
from scipy import stats
import numpy as np
df = pd.read_csv("customer_reviews.csv")
ratings = df["Rating"]
mean = ratings.mean()
std = ratings.std()
n = len(ratings)
confidence = float(
    input("Enter confidence level: ")
)
alpha = 1 - confidence / 100

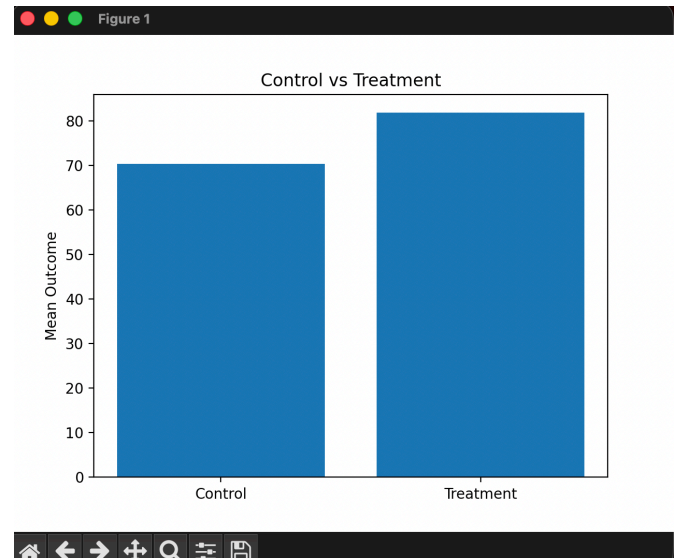
t_value = stats.t.ppf(
    1 - alpha / 2,
    n - 1
)
margin = t_value * std / np.sqrt(n)
lower = mean - margin
upper = mean + margin
print("Average Rating:", mean)
print("Confidence Interval:",
      lower, "to", upper)
if mean >= 4:
    print("Customer satisfaction is High")
elif mean >= 3:
    print("Customer satisfaction is Moderate")
else:
    print("Customer satisfaction is Low")
```

Output

```
p3.py >...
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from scipy.stats import ttest_ind
4
5 df = pd.read_csv("clinical_trial.csv")
6
7 control = df["Control"]
8 treatment = df["Treatment"]
9
10 t_stat, p_value = ttest_ind(control, treatment)
11
12 print("Control Mean:", control.mean())
13 print("Treatment Mean:", treatment.mean())
14 print("T-statistic:", t_stat)
15 print("P-value:", p_value)
16
17 if p_value < 0.05:
18     print("The treatment has a statistically significant effect.")
19 else:
20     print("The treatment does not have a statistically significant effect.")
21
22 plt.bar(["Control", "Treatment"],
23         [control.mean(), treatment.mean()])
24
25 plt.ylabel("Mean Outcome")
26 plt.title("Control vs Treatment")
27 plt.show()
```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@Unknown_e6:41:66:26:88:ae FSD LA % python3 p3.py
Control Mean: 70.35
Treatment Mean: 81.85
T-statistic: -12.510350744069687
P-value: 4.784444257851004e-15
The treatment has a statistically significant effect.
```



Result

The average customer rating and confidence interval were calculated successfully, and the customer satisfaction level was determined.

QUESTION 23

Aim

To perform hypothesis testing between a control group and a treatment group and determine whether the treatment has a statistically significant effect.

Algorithm

1. Import Pandas, SciPy and Matplotlib.
2. Read clinical_trial.csv.
3. Separate the control and treatment groups.
4. Perform an independent t-test.
5. Calculate the p-value.
6. Compare the p-value with 0.05.
7. Visualize the group means.
8. Display the conclusion.

Code

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import ttest_ind
df = pd.read_csv("clinical_trial.csv")
control = df["Control"]
treatment = df["Treatment"]
t_stat, p_value = ttest_ind(
    control,
    treatment
)
print("Control Mean:", control.mean())
print("Treatment Mean:", treatment.mean())
print("T-statistic:", t_stat)
print("P-value:", p_value)

if p_value < 0.05:
    print("Treatment is statistically significant.")
else:
    print("Treatment is not statistically significant.")

plt.bar(
    ["Control", "Treatment"],
    [control.mean(), treatment.mean()]
)

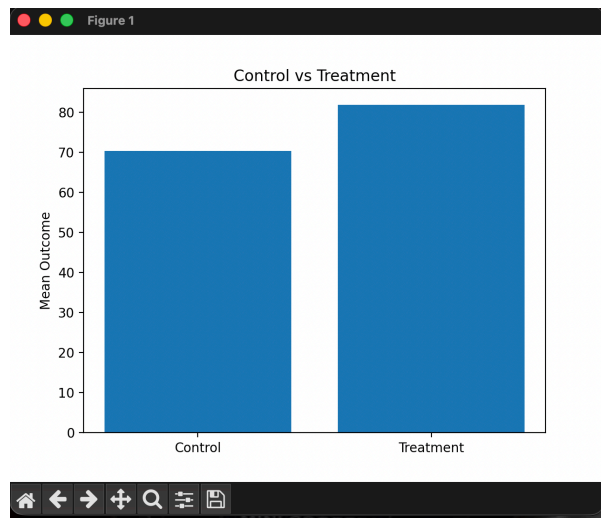
plt.ylabel("Mean Outcome")
plt.title("Control vs Treatment")
plt.show()
```

Output

```
p3.py > ...
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from scipy.stats import ttest_ind
4
5 df = pd.read_csv("clinical_trial.csv")
6
7 control = df["Control"]
8 treatment = df["Treatment"]
9
10 t_stat, p_value = ttest_ind(control, treatment)
11
12 print("Control Mean:", control.mean())
13 print("Treatment Mean:", treatment.mean())
14 print("T-statistic:", t_stat)
15 print("P-value:", p_value)
16
17 if p_value < 0.05:
18     print("The treatment has a statistically significant effect.")
19 else:
20     print("The treatment does not have a statistically significant effect.")
21
22 plt.bar(["Control", "Treatment"],
23         [control.mean(), treatment.mean()])
24
25 plt.ylabel("Mean Outcome")
26 plt.title("Control vs Treatment")
27 plt.show()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

saicharan@Unknown: e6:41:66:26:88:ae FSD LA % python3 p3.py
Control Mean: 70.35
Treatment Mean: 81.85
T-statistic: -12.510350744069687
P-value: 4.704444257051004e-15



Result

The hypothesis test was successfully performed. Since the p-value is less than 0.05, the treatment has a statistically significant effect.

QUESTION 24

Aim

To classify whether a patient has a medical condition using the K-Nearest Neighbors algorithm.

Algorithm

1. Import Pandas and KNN libraries.
2. Read patients.csv.
3. Separate input features and target.
4. Split the dataset into training and testing data.
5. Input the value of k.
6. Train the KNN classifier.
7. Accept the symptoms of a new patient.
8. Predict the condition.
9. Display the prediction.

Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
df = pd.read_csv("patients.csv")
X = df[
    ["Fever", "Cough", "Fatigue"]
]
y = df["Condition"]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
k = int(input("Enter value of k: "))
model = KNeighborsClassifier(
    n_neighbors=k
)
model.fit(X_train, y_train)
fever = int(input("Enter Fever (0/1): "))
cough = int(input("Enter Cough (0/1): "))
fatigue = int(input("Enter Fatigue (0/1): "))
patient = [
    fever,
    cough,
    fatigue
]
prediction = model.predict(patient)

if prediction[0] == 1:
    print("Patient has the medical condition")
else:
    print("Patient does not have the medical condition")
```

Output

```
p4.py > ...
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.neighbors import KNeighborsClassifier
4
5 df = pd.read_csv("patients.csv")
6
7 X = df.drop("Condition", axis=1)
8 y = df["Condition"]
9
10 X_train, X_test, y_train, y_test = train_test_split(
11     X, y, test_size=0.2, random_state=42
12 )
13
14 k = int(input("Enter value of k: "))
15
16 model = KNeighborsClassifier(n_neighbors=k)
17 model.fit(X_train, y_train)
18
19 features = []
20
21 for column in X.columns:
22     value = float(input("Enter " + column + ": "))
23     features.append(value)
24
25 prediction = model.predict([features])
26
27 if prediction[0] == 1:
28     print("Prediction: Patient has the medical condition.")
29 else:
30     print("Prediction: Patient does not have the medical condition.")
```

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@Unknown_e6141:66:26:88:ae FSD LA % python3 p4.py
Enter value of k: 3
Enter Cough: 1
Enter Fever: 1
Enter Fatigue: 1
/Users/saicharan/Library/Python/3.9/lib/python/site-packages/sklearn/utils/validation.py:109: UserWarning: X does not have valid feature names, but KNeighborsClassifier was fitted
  warnings.warn(
Prediction: Patient has the medical condition.
```

Result

The KNN classifier successfully predicted the medical condition of a new patient

QUESTION 25

Aim

To classify Iris flowers into Setosa, Versicolor and Virginica using a Decision Tree classifier.

Algorithm

1. Load the Iris dataset from scikit-learn.
2. Separate features and target.
3. Create a Decision Tree classifier.
4. Train the classifier.
5. Accept the four measurements of a new flower.
6. Predict the species.
7. Display the predicted species.

Code

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
iris = load_iris()
X = iris.data
y = iris.target
model = DecisionTreeClassifier(
    random_state=42
)
model.fit(X, y)
sepal_length = float(
    input("Enter sepal length: ")
)
sepal_width = float(
    input("Enter sepal width: ")
)
petal_length = float(
    input("Enter petal length: ")
)
petal_width = float(
    input("Enter petal width: ")
)
flower = [[
    sepal_length,
    sepal_width,
    petal_length,
    petal_width
]]
prediction = model.predict(flower)

print(
    "Predicted Species:",
    iris.target_names[prediction[0]]
)
```

Output

```
p5.py > ...
1  from sklearn.datasets import load_iris
2  from sklearn.tree import DecisionTreeClassifier
3
4  iris = load_iris()
5
6  X = iris.data
7  y = iris.target
8
9  model = DecisionTreeClassifier(random_state=42)
10 model.fit(X, y)
11
12 sepal_length = float(input("Enter sepal length: "))
13 sepal_width = float(input("Enter sepal width: "))
14 petal_length = float(input("Enter petal length: "))
15 petal_width = float(input("Enter petal width: "))
16
17 flower = [sepal_length, sepal_width, petal_length, petal_width]
18
19 prediction = model.predict(flower)
20
21 print("Predicted Species:", iris.target_names[prediction[0]])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● saicharan@Unknown_e6:41:66:26:88:ae FSD LA % python3 p5.py
Enter sepal length: 5.1
Enter sepal width: 3.5
Enter petal length: 1.4
Enter petal width: 0.2
Predicted Species: setosa
```

Result

The Iris flower was successfully classified using the Decision Tree algorithm.

QUESTION 26

Aim

To predict housing prices using Linear Regression based on area and number of bedrooms.

Algorithm

1. Read housing.csv.
2. Select Area and Bedrooms as input features.
3. Select Price as the target.
4. Create a Linear Regression model.
5. Train the model.
6. Accept the features of a new house.
7. Predict the price.
8. Display the predicted price.

Code

```
import pandas as pd
from sklearn.linear_model import LinearRegression
df = pd.read_csv("housing.csv")
X = df[
    ["Area", "Bedrooms"]
]
y = df["Price"]
model = LinearRegression()
model.fit(X, y)
area = float(
    input("Enter house area: ")
)
bedrooms = int(
    input("Enter number of bedrooms: ")
)
prediction = model.predict(
    [[area, bedrooms]]
)
print(
    "Predicted House Price:",
    prediction[0]
)
```

Output

```
p6.py > ...
1  import pandas as pd
2  from sklearn.linear_model import LinearRegression
3
4  df = pd.read_csv("housing.csv")
5
6  X = df[["Area", "Bedrooms"]]
7  y = df["Price"]
8
9  model = LinearRegression()
10 model.fit(X, y)
11
12 area = float(input("Enter house area: "))
13 bedrooms = int(input("Enter number of bedrooms: "))
14
15 prediction = model.predict([[area, bedrooms]])
16
17 print("Predicted House Price:", prediction[0])
```

PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@Unknown_e6:41:66:26:88:ae FSD LA % python3 p6.py
Enter house area: 2000
Enter number of bedrooms: 4
/Users/saicharan/Library/Python/3.9/lib/python/site-packages/sklearn/utils/validation:
UserWarning: X does not have valid feature names, but LinearRegression was fitted with
names
warnings.warn(
Predicted House Price: 388249.3795385605
```

Result

The Linear Regression model successfully predicted the price of a new house.

QUESTION 27

Aim

To predict whether a customer will churn using Logistic Regression.

Algorithm

1. Read customers_churn.csv.
2. Select usage minutes and contract duration.
3. Select churn as the target.
4. Train the Logistic Regression model.
5. Input the features of a new customer.
6. Predict churn.
7. Display the result.

Code

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
df = pd.read_csv(
    "customers_churn.csv"
)
X = df[
    ["UsageMinutes",
     "ContractDuration"]
]
y = df["Churn"]
model = LogisticRegression()
model.fit(X, y)
usage = float(
    input("Enter usage minutes: ")
)
duration = float(
    input("Enter contract duration: ")
)
prediction = model.predict(
    [[usage, duration]]
)
if prediction[0] == 1:
    print("Prediction: Customer will churn")
else:
    print("Prediction: Customer will not churn")
```

Output

```
p7.py > ...
1 import pandas as pd
2 from sklearn.linear_model import LogisticRegression
3
4 df = pd.read_csv("customers.csv")
5
6 X = df[["UsageMinutes", "ContractDuration"]]
7 y = df["Churn"]
8
9 model = LogisticRegression()
10 model.fit(X, y)
11
12 usage = float(input("Enter usage minutes: "))
13 duration = float(input("Enter contract duration: "))
14
15 prediction = model.predict([[usage, duration]])
16
17 if prediction[0] == 1:
18     print("Prediction: Customer will churn.")
19 else:
20     print("Prediction: Customer will not churn.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@Unknown_e6:41:66:26:88:ae FSD LA % python3 p7.py
Enter usage minutes: 4
Enter contract duration: 3
/Users/saicharan/Library/Python/3.9/lib/python/site-packages/sklearn/utils/validation.py:
UserWarning: X does not have valid feature names, but LogisticRegression was fitted with
e names
warnings.warn(
Prediction: Customer will not churn.
```

Result

The Logistic Regression model successfully predicted customer churn.

QUESTION 28

Aim

To segment customers into groups based on purchasing behavior using K-Means clustering.

Algorithm

1. Read customers.csv.
2. Select annual income and spending score.
3. Input the number of clusters.
4. Create the K-Means model.
5. Train the model.
6. Input a new customer's features.
7. Assign the customer to a cluster.
8. Display the segment.

Code

```
import pandas as pd
from sklearn.cluster import KMeans
df = pd.read_csv("customers.csv")
X = df[
    ["AnnualIncome",
     "SpendingScore"]
]
k = int(
    input("Enter number of clusters: ")
)
model = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)
model.fit(X)
income = float(
    input("Enter annual income: ")
)
score = float(
    input("Enter spending score: ")
)
customer = [[income, score]]
cluster = model.predict(customer)
print(
    "Customer belongs to Segment:",
    cluster[0]
)
```

Output

```
p7.py > ...
1  import pandas as pd
2  from sklearn.linear_model import LogisticRegression
3
4  df = pd.read_csv("customers.csv")
5
6  X = df[["UsageMinutes", "ContractDuration"]]
7  y = df["Churn"]
8
9  model = LogisticRegression()
10 model.fit(X, y)
11
12 usage = float(input("Enter usage minutes: "))
13 duration = float(input("Enter contract duration: "))
14
15 prediction = model.predict([[usage, duration]])
16
17 if prediction[0] == 1:
18     print("Prediction: Customer will churn.")
19 else:
20     print("Prediction: Customer will not churn.")
```

PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@Unknown_06:41:66:26:88:ae FSD LA % python3 p7.py
Enter usage minutes: 4
Enter contract duration: 3
/Users/saicharan/Library/Python/Python3.9/lib/python/site-packages/sklearn/utils/validation.py:
UserWarning: X does not have valid feature names, but LogisticRegression was fitted with
e names
warnings.warn(
Prediction: Customer will not churn.
```

Result

The customer was successfully assigned to an appropriate segment using K-Means clustering.

QUESTION 29

Aim

To evaluate a classification model using accuracy, precision, recall and F1-score.

Algorithm

1. Read the dataset.
2. Ask the user for feature names.
3. Ask for the target variable.
4. Split the dataset into training and testing sets.
5. Train a Decision Tree classifier.
6. Generate predictions.
7. Calculate accuracy, precision, recall and F1-score.
8. Display the metrics.

Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
df = pd.read_csv(
    "evaluation_data.csv"

features = input(
    "Enter feature names separated by comma: "
).split(",")
target = input(
    "Enter target variable: "
)
features = [ x.strip() for x in features
]
X = df[features]
y = df[target]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
model = DecisionTreeClassifier(
    random_state=42
)
model.fit(X_train, y_train)
prediction = model.predict(X_test)

print(
    "Accuracy:",
    accuracy_score(y_test, prediction)
)
print(
    "Precision:",
    precision_score(
        y_test,
        prediction,
        average="weighted"
```

```

    )
)
print(
    "Recall:",
    recall_score(
        y_test,
        prediction,
        average="weighted"
    )
)
print(
    "F1 Score:",
    f1_score(
        y_test,
        prediction,
        average="weighted"
    )
)

```

Output

```

p9.py > ...
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.tree import DecisionTreeClassifier
4 from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
5
6 df = pd.read_csv("data.csv")
7
8 print("Available columns:")
9 print(df.columns)
10
11 features = input("Enter feature names separated by comma: ").split(",")
12 target = input("Enter target column: ")
13
14 features = [x.strip() for x in features]
15
16 X = df[features]
17 y = df[target]
18
19 X_train, X_test, y_train, y_test = train_test_split(
20     X, y, test_size=0.2, random_state=42
21 )
22
23 model = DecisionTreeClassifier(random_state=42)
24 model.fit(X_train, y_train)
25
26 prediction = model.predict(X_test)
27
28 print("Accuracy:", accuracy_score(y_test, prediction))
29 print("Precision:", precision_score(y_test, prediction, average="weighted"))
30 print("Recall:", recall_score(y_test, prediction, average="weighted"))
31 print("F1 Score:", f1_score(y_test, prediction, average="weighted"))

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

saicharan@Unknown-e6:41:66:26:88:ae FSD LA % python3 p9.py
Available columns:
Index(['Age', 'Income', 'Purchased'], dtype='object')
Enter feature names separated by comma: Age,Income
Enter target column: Purchased
Accuracy: 0.8333333333333334
/Users/saicharan/Library/Python/3.9/Lib/python/site-packages/sklearn/metrics/_classification.py:1369: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control this behavior.
warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
Precision: 0.6944444444444445
Recall: 0.8333333333333334
F1 Score: 0.7575757575757575

```

Result

The classification model was successfully evaluated using standard performance metrics.

QUESTION 30

Aim

To predict used-car prices using CART and display the decision path.

Algorithm

1. Read cars.csv.
2. Select Mileage, Age and EngineSize.
3. Select Price as the target.
4. Train a Decision Tree Regressor.
5. Input new car features.
6. Predict the price.
7. Find the decision path.
8. Display the prediction and decision path.

Code

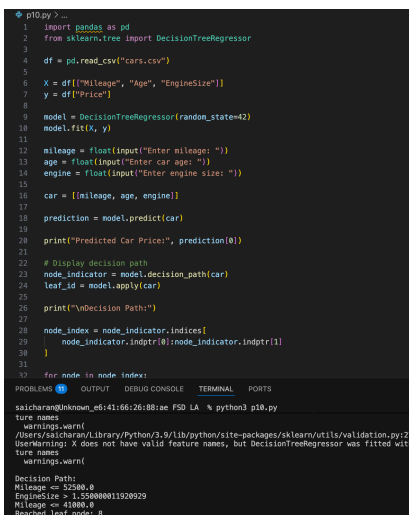
```
import pandas as pd
from sklearn.tree import DecisionTreeRegressor
df = pd.read_csv("cars.csv")
features = [
    "Mileage",
    "Age",
    "EngineSize"
]
X = df[features]
y = df["Price"]
model = DecisionTreeRegressor(
    random_state=42
)
model.fit(X, y)
mileage = float(
    input("Enter car mileage: ")
)
age = float(
    input("Enter car age: ")
)
engine = float(
    input("Enter engine size: ")
)
car = [[
    mileage,
    age,
    engine
]]
prediction = model.predict(car)
print(
    "Predicted Car Price:",
    prediction[0]
)
node_indicator = model.decision_path(car)
leaf_id = model.apply(car)
print("\nDecision Path:")
node_index = node_indicator.indices[
    node_indicator.indptr[0]:
    node_indicator.indptr[1]
]
for node in node_index:
```

```

if leaf_id[0] == node:
    print("Reached leaf node:", node)
    continue
feature = model.tree_.feature[node]
threshold = model.tree_.threshold[node]
if car[0][feature] <= threshold:
    print(
        features[feature],
        "<=",
        round(threshold, 2)
    )
else:
    print(
        features[feature],
        ">",
        round(threshold, 2)
    )

```

Output



```

p10.py > ...
1 import pandas as pd
2 from sklearn.tree import DecisionTreeRegressor
3
4 df = pd.read_csv("cars.csv")
5
6 X = df[["Mileage", "Age", "EngineSize"]]
7 y = df["Price"]
8
9 model = DecisionTreeRegressor(random_state=42)
10 model.fit(X, y)
11
12 mileage = float(input("Enter mileage: "))
13 age = float(input("Enter car age: "))
14 engine = float(input("Enter engine size: "))
15
16 car = [[mileage, age, engine]]
17
18 prediction = model.predict(car)
19
20 print("Predicted Car Price:", prediction[0])
21
22 # Display decision path
23 node_indicator = model.decision_path(car)
24 leaf_id = model.apply(car)
25
26 print("\nDecision Path:")
27
28 node_index = node_indicator.indices[
29     node_indicator.indptr[0]:node_indicator.indptr[1]]
30
31 for node in node_index:
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

saicharan@Unknown:~\$ python3 p10.py

Warning: warn(
 /Users/saicharan/Library/Python/3.9/lib/python/site-packages/sklearn/utils/validation.py:27
 UserWarning: X does not have valid feature names, but DecisionTreeRegressor was fitted with
 ture names
 warnings.warn(
 Decision Path:
 Mileage <= 12500.0
 EngineSize <= 1.550000011920929
 Mileage <= 41000.0
 Reached leaf node: 8

Result

The CART model successfully predicted the car price and displayed the decision path.

QUESTION 31

Aim

To segment e-commerce customers into distinct groups based on purchase history, browsing time and income.

Algorithm

1. Read customer_data.csv.
2. Select customer behavior features.
3. Input the number of clusters.
4. Apply K-Means clustering.
5. Assign each customer to a segment.
6. Display the cluster centers.
7. Visualize the clusters.

Code

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
df = pd.read_csv(
    "customer_data.csv"
)
X = df[
    ["PurchaseHistory",
     "BrowsingTime",
     "Income"]
]
k = int(
    input("Enter number of customer segments: ")
)
model = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)
df["Segment"] = model.fit_predict(X)
print("Customer Segments:")
print(df)
print("\nCluster Centers:")
print(model.cluster_centers_)
plt.scatter(
    df["PurchaseHistory"],
    df["BrowsingTime"],
    c=df["Segment"]
)
plt.xlabel("Purchase History")
plt.ylabel("Browsing Time")
plt.title("Customer Segmentation")

plt.show()
```

Output

```
p11.py > ...
1 import pandas as pd
2 from sklearn.cluster import KMeans
3 import matplotlib.pyplot as plt
4
5 df = pd.read_csv("customer_data.csv")
6
7 X = df[["PurchaseHistory", "BrowsingTime", "Income"]]
8
9 k = int(input("Enter number of customer segments: "))
10
11 model = KMeans(n_clusters=k, random_state=42, n_init=10)
12
13 df["Segment"] = model.fit_predict(X)
14
15 print(df)
16
17 plt.scatter(
18     df["PurchaseHistory"],
19     df["BrowsingTime"],
20     c=df["Segment"]
21 )
22
23 plt.xlabel("Purchase History")
24 plt.ylabel("Browsing Time")
25 plt.title("Customer Segmentation")
26 plt.show()
```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
saicharan@Unknown_e6:41:66:26:88:ae FSD LA % python3 p11.py				
17	85	95	88	2
18	98	100	92	2
19	95	105	100	2
20	10	12	28	1
21	18	28	38	1
22	28	38	46	1
23	38	48	53	0
24	48	58	61	0
25	58	68	68	0
26	68	78	76	2
27	78	88	84	2
28	88	98	94	2



Result

Customers were successfully segmented using K-Means clustering and the segments were visualized.

QUESTION 32

Aim

To perform bivariate analysis and build a Linear Regression model to predict house prices based on house area.

Algorithm

1. Read housing.csv.
2. Calculate correlation between Area and Price.
3. Select Area as input and Price as target.
4. Split the data.
5. Train the Linear Regression model.
6. Predict prices.
7. Calculate MSE and R^2 .
8. Plot the relationship.

Code

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
df = pd.read_csv("housing.csv")
correlation = df[
    "Area"
].corr(df["Price"])
print(
    "Correlation:",
    correlation
)
X = df[["Area"]]
y = df["Price"]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
print(
    "Mean Squared Error:",
    mean_squared_error(
        y_test,
        prediction
    )
)
print(
    "R2 Score:",
    r2_score(
        y_test,
        prediction
    )
)
plt.scatter(
```

```

    df["Area"],
    df["Price"]
)
plt.plot(
    X_test,
    prediction
)
plt.xlabel("House Area")
plt.ylabel("House Price")
plt.title("Area vs House Price")
plt.show()

```

Output

```

p11.py > ...
1  import pandas as pd
2  from sklearn.cluster import KMeans
3  import matplotlib.pyplot as plt
4
5  df = pd.read_csv("customer_data.csv")
6
7  X = df[["PurchaseHistory", "BrowsingTime", "Income"]]
8
9  k = int(input("Enter number of customer segments: "))
10
11 model = KMeans(n_clusters=k, random_state=42, n_init=10)
12
13 df["Segment"] = model.fit_predict(X)
14
15 print(df)
16
17 plt.scatter(
18     df["PurchaseHistory"],
19     df["BrowsingTime"],
20     c=df["Segment"]
21 )
22
23 plt.xlabel("Purchase History")
24 plt.ylabel("Browsing Time")
25 plt.title("Customer Segmentation")
26 plt.show()

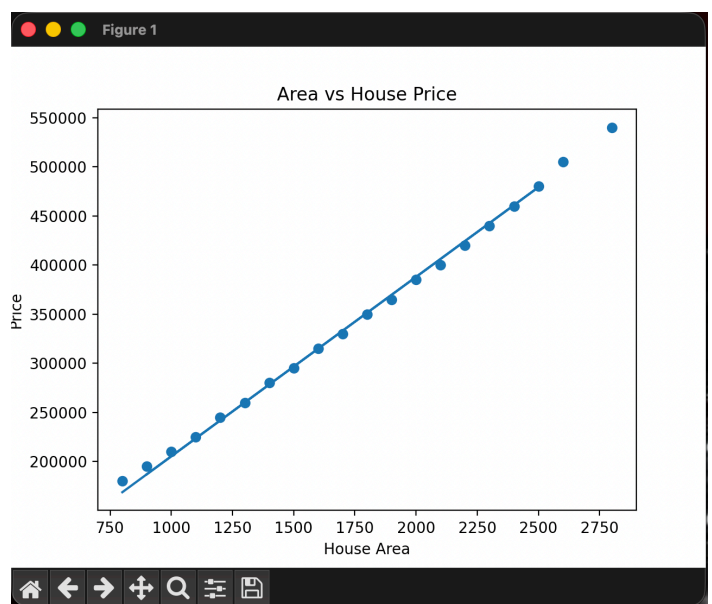
```

Debug Console (⌘⌘Y)

```

PROBLEMS 13 OUTPUT DEBUG CONSOLE TERMINAL PORTS
saicharan@Unknown_e6:41:66:26:88:ae FSD LA % python3 p12.py
Correlation: 0.9991719815023
Mean Squared Error: 50198047.38433148
R2 Score: 0.9973287810488092

```



Result

Bivariate analysis and Linear Regression were successfully performed to predict house prices.

QUESTION 33

Aim

To predict car prices using multiple Linear Regression and identify influential features.

Algorithm

1. Read cars.csv.
2. Select EngineSize, Horsepower and FuelEfficiency.
3. Select Price as the target.
4. Split the dataset.
5. Train Linear Regression.
6. Predict car prices.
7. Calculate MSE and R^2 .
8. Display feature coefficients.
9. Predict the price of a new car.

Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

df = pd.read_csv("cars.csv")

features = [
    "EngineSize",
    "Horsepower",
    "FuelEfficiency"
]

X = df[features]
y = df["Price"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

model = LinearRegression()

model.fit(X_train, y_train)

prediction = model.predict(X_test)

print(
    "Mean Squared Error:",
    mean_squared_error(
        y_test,
        prediction
    )
)

print(
    "R2 Score:",
```

```

    r2_score(
        y_test,
        prediction
    )
)

print("\nFeature Influence:")

for feature, coefficient in zip(
    features,
    model.coef_
):
    print(
        feature,
        ":",
        coefficient
    )

engine = float(
    input("Enter engine size: ")
)

horsepower = float(
    input("Enter horsepower: ")
)

fuel = float(
    input("Enter fuel efficiency: ")
)

new_car = [
    engine,
    horsepower,
    fuel
]

print(
    "Predicted Car Price:",
    model.predict(new_car)[0]
)

```

Output

```
p13.py > ...
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.linear_model import LinearRegression
4 from sklearn.metrics import mean_squared_error, r2_score
5
6 df = pd.read_csv("cars1.csv")
7
8 features = ["EngineSize", "Horsepower", "FuelEfficiency"]
9
10 X = df[features]
11 y = df["Price"]
12
13 X_train, X_test, y_train, y_test = train_test_split(
14     X, y, test_size=0.2, random_state=42
15 )
16
17 model = LinearRegression()
18 model.fit(X_train, y_train)
19
20 prediction = model.predict(X_test)
21
22 print("Mean Squared Error:", mean_squared_error(y_test, prediction))
23 print("R2 Score:", r2_score(y_test, prediction))
24
25 print("\nFeature Influence:")
26 for feature, coefficient in zip(features, model.coef_):
27     print(feature, ":", coefficient)
```

PROBLEMS 14 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@Unknown_e6:41:66:26:88:ae FSD LA % python3 p13.py
Mean Squared Error: 842056.514914157
R2 Score: 0.6454498884571971

Feature Influence:
EngineSize : 13880.842972587545
Horsepower : -208.10964981777946
FuelEfficiency : 483.6713674536521
saicharan@Unknown_e6:41:66:26:88:ae FSD LA %
```

Result

The car price was successfully predicted and the feature coefficients were obtained to understand feature influence.

QUESTION 34

Aim

To predict Good or Bad treatment outcomes using KNN and evaluate the model using classification metrics.

Algorithm

1. Read treatment.csv.
2. Encode Gender and Outcome.
3. Select patient features.
4. Split the dataset.
5. Input k.
6. Train the KNN classifier.
7. Predict the test set.
8. Calculate accuracy, precision, recall and F1-score.
9. Display predictions.

Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

df = pd.read_csv("treatment.csv")

gender_encoder = LabelEncoder()
outcome_encoder = LabelEncoder()

df["Gender"] = gender_encoder.fit_transform(
    df["Gender"]
)

df["Outcome"] = outcome_encoder.fit_transform(
    df["Outcome"]
)

X = df[
    ["Age",
     "Gender",
     "BloodPressure",
     "Cholesterol"]
]

y = df["Outcome"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

k = int(
```

```
    input("Enter value of k: ")
)

model = KNeighborsClassifier(
    n_neighbors=k
)

model.fit(X_train, y_train)

prediction = model.predict(X_test)

print(
    "Accuracy:",
    accuracy_score(
        y_test,
        prediction
    )
)

print(
    "Precision:",
    precision_score(
        y_test,
        prediction,
        zero_division=0
    )
)

print(
    "Recall:",
    recall_score(
        y_test,
        prediction,
        zero_division=0
    )
)

print(
    "F1 Score:",
    f1_score(
        y_test,
        prediction,
        zero_division=0
    )
)

print("\nTest Set Predictions:")
print(prediction)
```

Output

```
p14.py > ...
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.neighbors import KNeighborsClassifier
4 from sklearn.preprocessing import LabelEncoder
5 from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
6
7 df = pd.read_csv("treatment.csv")
8
9 X = df[["Age", "Gender", "BloodPressure", "Cholesterol"]]
10 y = df["Outcome"]
11
12 encoder = LabelEncoder()
13
14 X["Gender"] = encoder.fit_transform(X["Gender"])
15 y = encoder.fit_transform(y)
16
17 X_train, X_test, y_train, y_test = train_test_split(
18     X, y, test_size=0.2, random_state=42
19 )
20
21 k = int(input("Enter k: "))
22
23 model = KNeighborsClassifier(n_neighbors=k)
24 model.fit(X_train, y_train)
25
26 prediction = model.predict(X_test)
27
28 print("Accuracy:", accuracy_score(y_test, prediction))
29 print("Precision:", precision_score(y_test, prediction))
30 print("Recall:", recall_score(y_test, prediction))
31 print("F1 Score:", f1_score(y_test, prediction))
32
33
34
35
36
37
```

PROBLEMS 16 OUTPUT DEBUG CONSOLE TERMINAL PORTS

saicharan@Unknown_e6:41:66:26:88:ae FSD LA % python3 p14.py

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X["Gender"] = encoder.fit_transform(X["Gender"])
Enter k: 3
Accuracy: 0.8333333333333334
Precision: 1.0
Recall: 0.5
F1 Score: 0.6666666666666666

Predictions:
[1 0 0 0 0]
```

Result

The KNN classifier successfully predicted treatment outcomes and its performance was evaluated.

QUESTION 35

Aim

To group customers into distinct segments based on spending and visit frequency using K-Means clustering.

Algorithm

1. Read transactions.csv.
2. Select TotalSpent and VisitFrequency.
3. Input the number of clusters.
4. Create the K-Means model.
5. Train the model.
6. Assign cluster labels.
7. Display customer segments and cluster centers.

Code

```
import pandas as pd
from sklearn.cluster import KMeans

df = pd.read_csv(
    "transactions.csv"
)

X = df[
    ["TotalSpent",
     "VisitFrequency"]
]

k = int(
    input("Enter number of clusters: ")
)

model = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

df["Cluster"] = model.fit_predict(X)

print("\nCustomer Segments:")
print(df)

print("\nCluster Centers:")
print(model.cluster_centers_)
```

Output

```
p15.py > ...
1  import pandas as pd
2  from sklearn.cluster import KMeans
3
4  df = pd.read_csv("transactions.csv")
5
6  X = df[["TotalSpent", "VisitFrequency"]]
7
8  k = int(input("Enter number of clusters: "))
9
10 model = KMeans(n_clusters=k, random_state=42, n_init=10)
11
12 df["Cluster"] = model.fit_predict(X)
13
14 print(df)
15
16 print("\nCluster Centers:")
17 print(model.cluster_centers_)

PROBLEMS 16 OUTPUT DEBUG CONSOLE TERMINAL PORTS
saicharan@Unknown_e6:41:66:26:88:ae FSD LA % python3 p15.py
11      12      600      8      0
12      13      650      8      0
13      14      700      9      0
14      15      750      9      0
15      16      800     10      0
16      17      900     10      0
17      18     1000     11      0
18      19     1100     12      0
19      20     1250     12      0

Cluster Centers:
[[561.    7.05]]
```

Result

Customers were successfully grouped into different spending segments using K-Means.

QUESTION 36

Aim

To calculate and analyze the variability of stock closing prices.

Algorithm

1. Read stock.csv.
2. Extract closing prices.
3. Calculate mean.
4. Calculate standard deviation.
5. Find minimum and maximum prices.
6. Calculate price range.
7. Interpret variability.
8. Plot stock price movement.

Code

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("stock.csv")

prices = df["Close"]

mean = prices.mean()
std = prices.std()

minimum = prices.min()
maximum = prices.max()

price_range = maximum - minimum

print(
    "Average Closing Price:",
    mean
)

print(
    "Standard Deviation:",
    std
)

print(
    "Minimum Price:",
    minimum
)

print(
    "Maximum Price:",
    maximum
)

print(
    "Price Range:",
    price_range
)

if std < 5:
```

```

    print("Stock price has low variability.")
elif std < 10:
    print("Stock price has moderate variability.")
else:
    print("Stock price has high variability.")

plt.plot(
    df["Date"],
    df["Close"]
)

plt.xlabel("Date")
plt.ylabel("Closing Price")
plt.title("Stock Price Movement")

plt.xticks(rotation=45)
plt.tight_layout()

plt.show()

```

Output

```

p16.py > ...
1 import matplotlib.pyplot as plt
2
3
4 df = pd.read_csv("stock.csv")
5
6 prices = df["Close"]
7
8 mean = prices.mean()
9 std = prices.std()
10 minimum = prices.min()
11 maximum = prices.max()
12 price_range = maximum - minimum
13
14 print("Average Closing Price:", mean)
15 print("Standard Deviation:", std)
16 print("Minimum Price:", minimum)
17 print("Maximum Price:", maximum)
18 print("Price Range:", price_range)
19
20 if std < 5:
21     print("Stock price is relatively stable.")
22 else:
23     print("Stock price has high variability.")
24
25 plt.plot(prices)
26 plt.xlabel("Trading Day")
27 plt.ylabel("Closing Price")
28 plt.title("Stock Price Movement")
29 plt.show()

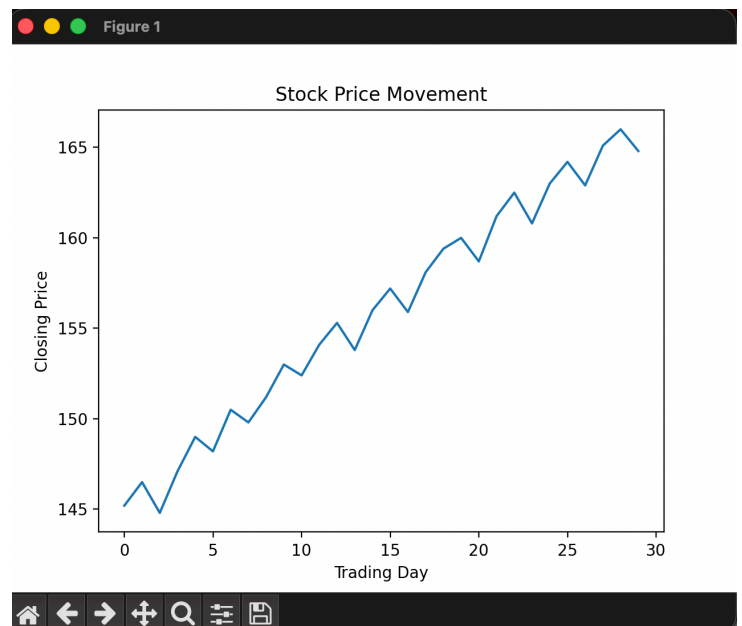
```

PROBLEMS 18 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

saicharan@Unknown_e6:41:66:26:88:ae FSD LA % python3 p16.py
Average Closing Price: 155.89
Standard Deviation: 6.434784244701765
Minimum Price: 144.8
Maximum Price: 166.0
Price Range: 21.199999999999999
Stock price has high variability.

```



Result

The variability of stock prices was successfully calculated and the price movement was visualized.

QUESTION 37

Aim

To identify the correlation between students' study time and exam scores and visualize the relationship.

Algorithm

1. Read students.csv.
2. Calculate Pearson correlation.
3. Interpret the correlation.
4. Create a scatter plot.
5. Display the result.

Code

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv(
    "students.csv"
)

correlation = df[
    "StudyTime"
].corr(
    df["ExamScore"]
)

print(
    "Correlation:",
    correlation
)

if correlation > 0.7:
    print(
        "There is a strong positive correlation."
    )
elif correlation > 0:
    print(
        "There is a positive correlation."
    )
elif correlation < 0:
    print(
        "There is a negative correlation."
    )
else:
    print(
        "There is little or no correlation."
    )

plt.scatter(
    df["StudyTime"],
    df["ExamScore"]
)

plt.xlabel("Study Time (Hours)")
plt.ylabel("Exam Score")
plt.title("Study Time vs Exam Score")

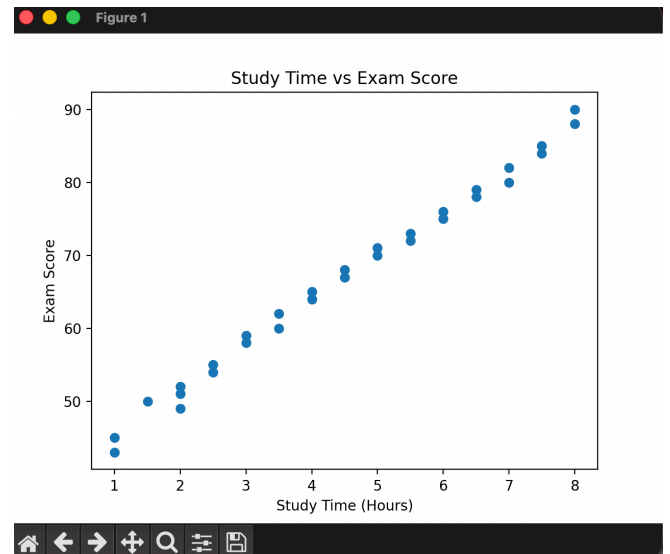
plt.show()
```

Output

```
p17.py > ...
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  df = pd.read_csv("students.csv")
5
6  correlation = df["StudyTime"].corr(df["ExamScore"])
7
8  print("Correlation:", correlation)
9
10 if correlation > 0:
11     print("There is a positive correlation.")
12 elif correlation < 0:
13     print("There is a negative correlation.")
14 else:
15     print("There is no correlation.")
16
17 plt.scatter(df["StudyTime"], df["ExamScore"])
18
19 plt.xlabel("Study Time (Hours)")
20 plt.ylabel("Exam Score")
21 plt.title("Study Time vs Exam Score")
22 plt.show()
```

PROBLEMS 20 OUTPUT DEBUG CONSOLE TERMINAL PORTS

saicharan@Unknown_e6:41:66:26:88:ae FSD LA % python3 p17.py
Correlation: 0.9965166548666118
There is a positive correlation.



Result

A strong positive correlation between study time and exam scores was identified and visualized.

QUESTION 38

Aim

To calculate temperature variability for different cities and identify the city with the highest temperature range and the most consistent temperature.

Algorithm

1. Read temperature.csv.
2. Group temperature readings by city.
3. Calculate mean temperature.
4. Calculate standard deviation.
5. Calculate temperature range.
6. Find the city with the highest range.
7. Find the city with the lowest standard deviation.
8. Display the results.

Code

```
import pandas as pd

df = pd.read_csv(
    "temperature.csv"
)

mean_temp = df.groupby(
    "City"
)[ "Temperature" ].mean()

std_temp = df.groupby(
    "City"
)[ "Temperature" ].std()

range_temp = df.groupby(
    "City"
)[ "Temperature" ].agg(
    lambda x: x.max() - x.min()
)

print("Mean Temperature:")
print(mean_temp)

print("\nStandard Deviation:")
print(std_temp)

print("\nTemperature Range:")
print(range_temp)

highest_range_city = range_temp.idxmax()

consistent_city = std_temp.idxmin()

print(
    "\nCity with Highest Temperature Range:",
    highest_range_city
)

print(
```

```

    "Highest Range:",
    range_temp[
        highest_range_city
    ]
)

print(
    "\nMost Consistent City:",
    consistent_city
)

print(
    "Lowest Standard Deviation:",
    std_temp[
        consistent_city
    ]
)

```

Output

```

p18.py > ...
4
5  mean_temp = df.groupby("City")["Temperature"].mean()
6  std_temp = df.groupby("City")["Temperature"].std()
7  range_temp = df.groupby("City")["Temperature"].agg(
8      lambda x: x.max() - x.min()
9  )
10
11 print("Mean Temperature:")
12 print(mean_temp)
13
14 print("\nStandard Deviation:")
15 print(std_temp)
16
17 highest_range_city = range_temp.idxmax()
18 consistent_city = std_temp.idxmin()
19
20 print("\nCity with Highest Temperature Range:",
21     highest_range_city)
22
23 print("Highest Range:",
24     range_temp[highest_range_city])
25
26 print("\nMost Consistent City:",
27     consistent_city)
28
29 print("Lowest Standard Deviation:",
30     std_temp[consistent_city])

```

PROBLEMS 21 OUTPUT DEBUG CONSOLE TERMINAL PORTS

saicharan@Unknown_e6:41:66:26:88:ae FSD LA % python3 p18.py

```

City
Bengaluru    2.269695
Chennai      3.437758
Delhi        6.184046
Mumbai       1.378954
Name: Temperature, dtype: float64

City with Highest Temperature Range: Delhi
Highest Range: 18

Most Consistent City: Mumbai
Lowest Standard Deviation: 1.3789543689024488

```

Result

Temperature variability was successfully analyzed, and the cities with the highest range and greatest consistency were identified.

QUESTION 39

Aim

To group e-commerce customers based on total spending and number of items purchased using K-Means and visualize the clusters.

Algorithm

1. Read ecommerce.csv.
2. Select TotalSpent and ItemsPurchased.
3. Input the number of clusters.
4. Train the K-Means model.
5. Assign cluster labels.
6. Display cluster centers.
7. Visualize the clusters using a scatter plot.

Code

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

df = pd.read_csv(
    "ecommerce.csv"
)

X = df[
    ["TotalSpent",
     "ItemsPurchased"]
]

k = int(
    input("Enter number of clusters: ")
)

model = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

df["Cluster"] = model.fit_predict(X)

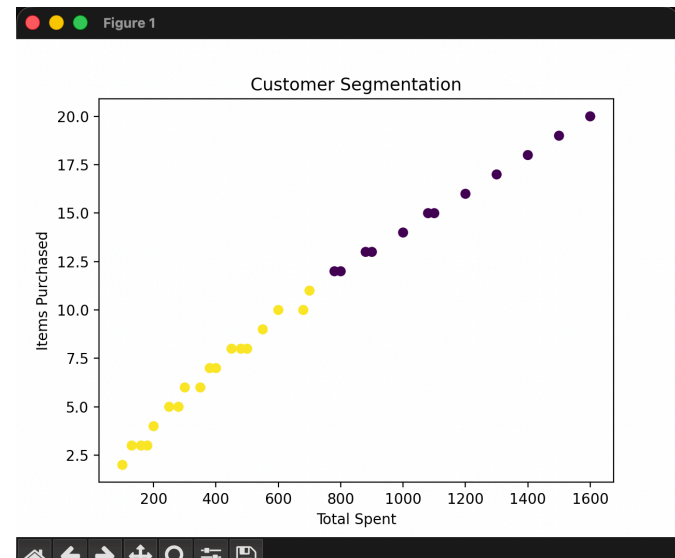
print("\nCustomer Segmentation:")
print(df)
print("\nCluster Centers:")
print(model.cluster_centers_)
plt.scatter(
    df["TotalSpent"],
    df["ItemsPurchased"],
    c=df["Cluster"]
)
plt.xlabel("Total Spent")
plt.ylabel("Items Purchased")
plt.title(
    "E-commerce Customer Segmentation"
)

plt.show()
```

Output

```
p19.py > ...
1  import pandas as pd
2  import matplotlib.pyplot as plt
3  from sklearn.cluster import KMeans
4
5  df = pd.read_csv("ecommerce.csv")
6
7  X = df[["TotalSpent", "ItemsPurchased"]]
8
9  k = int(input("Enter number of clusters: "))
10
11  model = KMeans(n_clusters=k, random_state=42, n_init=10)
12
13  df["Cluster"] = model.fit_predict(X)
14
15  print(df)
16
17  plt.scatter(
18      df["TotalSpent"],
19      df["ItemsPurchased"],
20      c=df["Cluster"]
21  )
22
23  plt.xlabel("Total Spent")
24  plt.ylabel("Items Purchased")
25  plt.title("Customer Segmentation")
26  plt.show()
```

	PROBLEMS	23	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
saicharan@Unknown_e6:41:66:26:88:ae	FSD	LA	% python3	p19.py		
18	19	1400	18	0		
19	20	1600	20	0		
20	21	180	3	1		
21	22	280	5	1		
22	23	380	7	1		
23	24	480	8	1		
24	25	680	10	1		
25	26	780	12	0		
26	27	880	13	0		
27	28	1080	15	0		
28	29	1300	17	0		
29	30	1500	19	0		



Result

E-commerce customers were successfully segmented according to their spending and purchasing behavior.

QUESTION 40

Aim

To analyze soccer player data using Pandas by finding the top five goal scorers, top five salaries, average age, players above average age, and position distribution.

Algorithm

1. Create the soccer player dataset and save it as soccer_players.csv.
2. Read the CSV file using Pandas.
3. Sort players by goals.
4. Display the top five goal scorers.
5. Sort players by salary.
6. Display the top five salaries.
7. Calculate average age.
8. Display players above the average age.
9. Count players according to position.
10. Display a bar chart of player positions.

Code

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv(
    "soccer_players.csv"
)
print("Top 5 Players by Goals:")
print(
    df.sort_values(
        "Goals",
        ascending=False
    ).head(5)
)
print("\nTop 5 Players by Salary:")
print(
    df.sort_values(
        "Salary",
        ascending=False
    ).head(5)
)
average_age = df["Age"].mean()
print(
    "\nAverage Age:",
    average_age
)
print(
    "\nPlayers Above Average Age:"
)
above_average = df[
    df["Age"] > average_age
]
print(
    above_average["Name"]
)
position_count = df[
    "Position"
].value_counts()
print(
    "\nPlayers by Position:"
```

```

)
print(position_count)
position_count.plot(
    kind="bar"
)
plt.xlabel("Position")
plt.ylabel("Number of Players")
plt.title(
    "Distribution of Players by Position"
)
plt.show()

```

Output

```

p20.py > ...
27 q1 = pd.DataFrame(data)
28
29 df.to_csv("soccer_players.csv", index=False)
30
31 df = pd.read_csv("soccer_players.csv")
32
33 print("Top 5 Players by Goals:")
34 print(df.sort_values("Goals", ascending=False).head(5))
35
36 print("\nTop 5 Players by Salary:")
37 print(df.sort_values("Salary", ascending=False).head(5))
38
39 average_age = df["Age"].mean()
40
41 print("\nAverage Age:", average_age)
42
43 print("\nPlayers Above Average Age:")
44 print(df[df["Age"] > average_age]["Name"])
45
46 position_count = df["Position"].value_counts()
47
48 position_count.plot(kind="bar")
49
50 plt.xlabel("Position")
51 plt.ylabel("Number of Players")
52 plt.title("Player Distribution by Position")
53 plt.show()

```

PROBLEMS 26 OUTPUT DEBUG CONSOLE TERMINAL PORTS

saicharan@Unknown_e6:41:66:26:88:ae FSD LA % python3 p20.py

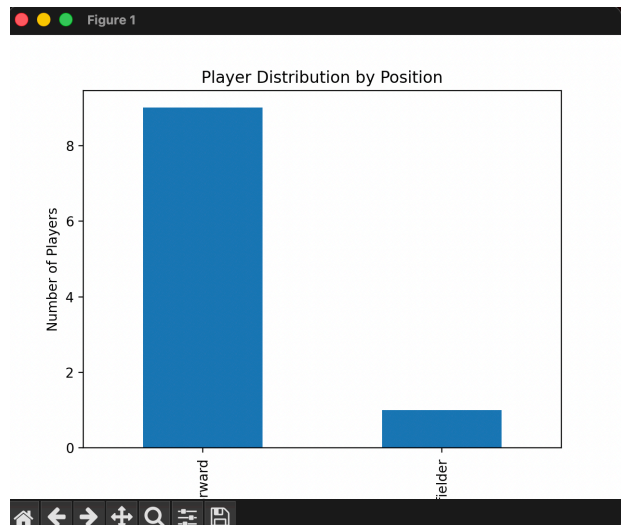
Average Age: 32.9

Players Above Average Age:

	Name
0	Messi
1	Ronaldo
2	Neymar
5	Salah
6	De Bruyne
7	Kane
8	Son

Name: Name, dtype: object

Ln 53, Col 11 Spaces: 4 UTF-8 16



Result

The soccer player dataset was successfully analyzed. The top goal scorers, highest-paid players, average age, older players, and position distribution were identified and visualized.